zkDPP 논문용 실험 결과 초안

실험 환경

항목	설정
Host PC	MacBook Pro (MacBookPro18,1)
Host OS	macOS 26.3 (25D125)
CPU	Apple M1 Pro, 10 cores (8 Performance + 2 Efficiency)
Host Memory	32 GB
Circuit 라이브러리	gnark 0.15.0 / gnark-crypto 0.20.1
Proof system	PLONK-KZG / BLS12-381 / Jubjub
Hash	Poseidon2
Blokchain Node	Anvil 1.7.1
Contract 컴파일러	Solidity 0.8.30
EVM	Prague

1. 공급망 Event별 실행 비용

실험을 진행한 이유
zkDPP의 Event는 공급망 동작에 더해 Proof를 검증하고 AuditRecord도 기록합니다.
이 작업들을 모두 수행하면, Event 하나를 실행하는 데 비용과 시간이 너무 많이 드는 것은 아닐까요?

실험 목적
9개 Event의 Constraints, 실제 transaction Gas, E2E를 측정합니다.

표·그래프 구성
Event 종류는 연속적인 증가 변수가 아니므로 정확한 값을 비교할 수 있는 표를 사용합니다.

고정 조건
모든 Supply Chain Event는 AuditRecord를 기록합니다.
State는 3개이며 Process의 입출력은 2→1입니다.

Event	Constraints	Gas	E2E (ms)
Entry	18,115	2,258,715	770.834
Transfer	49,883	3,577,414	1,464.369
Proceed	35,858	1,784,082	1,278.965
Recall	35,987	1,785,084	1,250.315
Merge	51,543	1,888,301	1,361.248
Split	48,608	2,730,046	1,497.729
Process	55,402	1,866,617	1,335.974
Exit	27,940	534,277	674.484
Issue	35,957	678,565	1,225.264

해석
비용의 핵심 증가 원인은 다음과 같습니다.

Constraints: Merkle Path 검증 + Encrypted AuditRecord 검증 + State 관계 검증 + cm/nf 계산
Gas: Merkle Tree Update + Proof 검증 + AuditRecord 저장 비용

Merkle Path 1회 검증, Merkle Tree 2회 Update를 진행하는 Transfer를 대표적인 Event로 선택해, Constraints와 Gas 소비를 분석합니다.

Audit과 관련된 추가 비용은 2.1절에서 추가로 분석했습니다.

(a) Merkle Path Verify

횟수	Event
0	Entry
1	Transfer / Proceed / Recall Split / Exit / Issue
2	Merge / Process

(b) Merkle Tree Update

횟수	Event
0	Exit / Issue
1	Entry / Proceed / Recall Merge / Process
2	Transfer / Split

Transfer의 Constraints 구성

비용 요인	Constraints
Merkle Path 검증	9,825
Encrypted AuditRecord 검증	14,238
State 범위 · 배분 등 검증	17,994
cm/nf · owner address 계산	7,826
Transfer 합계	**49,883**

Transfer의 Gas Cost 구성

비용 요인	Gas
Merkle Tree Hash 계산	1,422,404
Merkle Tree 조회 · 저장	1,276,500
Proof 검증 호출	335,007
Audit 추가 비용	356,146
nf · cm · rv 조회 · 저장	68,700
calldata 등 기타 비용	118,657
Transfer 합계	**3,577,414**

2. Audit 추가 비용과 Auditor 수행 비용

2.1 Event별 Audit 추가 비용

실험을 진행한 이유
zkDPP의 Event는 Auditing에 필요한 정보를 암호화해 기록합니다.
Audit이 없는 같은 Event보다 Constraints와 Gas가 크게 증가하는 것은 아닐까요?

표·그래프 구성
입력·State·권한·Tree 조건을 맞춘 Audit X/O 막대를 Event별로 나란히 배치합니다.
Constraints와 Gas는 단위가 다르므로 그래프를 분리하고, 표에는 각각의 증가량을 표시합니다.

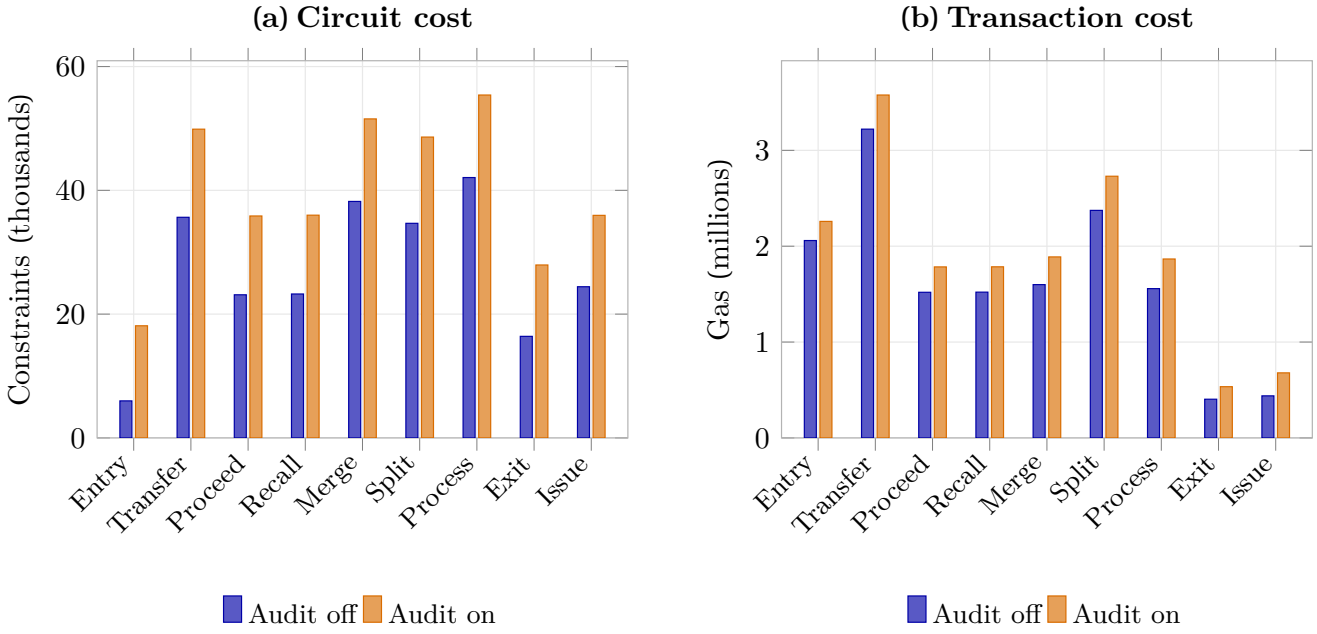


그림 1. 동일 Event의 Audit 유무에 따른 비용 비교.

(a) Constraints

Event	Audit X	Audit O	증가량
Entry	5,980	18,115	**12,135**
Transfer	35,645	49,883	**14,238**
Proceed	23,123	35,858	**12,735**
Recall	23,252	35,987	**12,735**
Merge	38,208	51,543	**13,335**
Split	34,671	48,608	**13,937**
Process	42,067	55,402	**13,335**
Exit	16,407	27,940	**11,533**
Issue	24,424	35,957	**11,533**

(b) Gas

Event	Audit X	Audit O	증가량
Entry	2,059,123	2,258,715	**199,592**
Transfer	3,221,268	3,577,414	**356,146**
Proceed	1,519,228	1,784,082	**264,854**
Recall	1,520,722	1,785,084	**264,362**
Merge	1,599,289	1,888,301	**289,012**
Split	2,373,759	2,730,046	**356,287**
Process	1,557,348	1,866,617	**309,269**
Exit	403,572	534,277	**130,705**
Issue	438,706	678,565	**239,859**

해석

Constraints 증가량은 약 1.1만–1.4만 개로, Event별로 비슷합니다.
Encrypted AuditRecord 검증에 필요한 R·Z의 scalar multiplication 두 번이 큰 공통 비용을 만듭니다.
암호화할 Field 수와 출력 nf·rvnf 계산에 따라 나머지 증가량이 달라집니다.

Gas 증가량은 약 13만–36만이며, Traceability 정보를 기록하는 SSTORE 비용이 크게 반영됩니다.
Contract는 Event가 만든 cm·rv·h마다 해당 Event의 AuditRecord ID를 기록합니다.
따라서 출력 cm·rv·h와 AuditRecord에 담는 암호화 값이 많아질수록 SSTORE 횟수도 증가합니다.

Transfer의 Audit 추가 Constraints

비용 요인	Constraints
R · Z scalar multiplication	8,611
키 · mask 계산	2,399
난수 · 암호문 등 검증	1,723
출력 nf · rvnf 계산	1,505
합계	**14,238**

Transfer의 Audit 추가 Gas

비용 요인	Gas
AuditRecord SSTORE	324,100
Proof 검증 호출 증가분	4,585
SLOAD · calldata 등 기타 증가분	27,461
합계	**356,146**

2.2 Tracing · Freeze 비용

실험을 진행한 이유
Tracing은 연결된 AuditRecord를 조회하고 복호화해야 합니다.
공급망이 길어져 방문할 기록이 많아지면, 추적에 너무 오래 걸리는 것은 아닐까요?
Freeze도 대상마다 transaction을 실행하므로, 동결 대상이 많아지면 시간과 Gas 부담이 커지는 것은 아닐까요?

표 · 그래프 구성
Tracing과 Freeze를 병렬로 배치하고, 같은 시간 단위와 로그축 범위로 비교합니다.
이 Split 시나리오에서는 방문 AuditRecord 수와 Freeze 대상 수가 각각 10 · 100 · 1,000개입니다.

측정 범위
Tracing은 기록 조회 · 복호화 · 연결 탐색 시간입니다.
Freeze는 제출 전 상태 검사부터 개별 동결 transaction · 최종 상태 확인까지입니다.

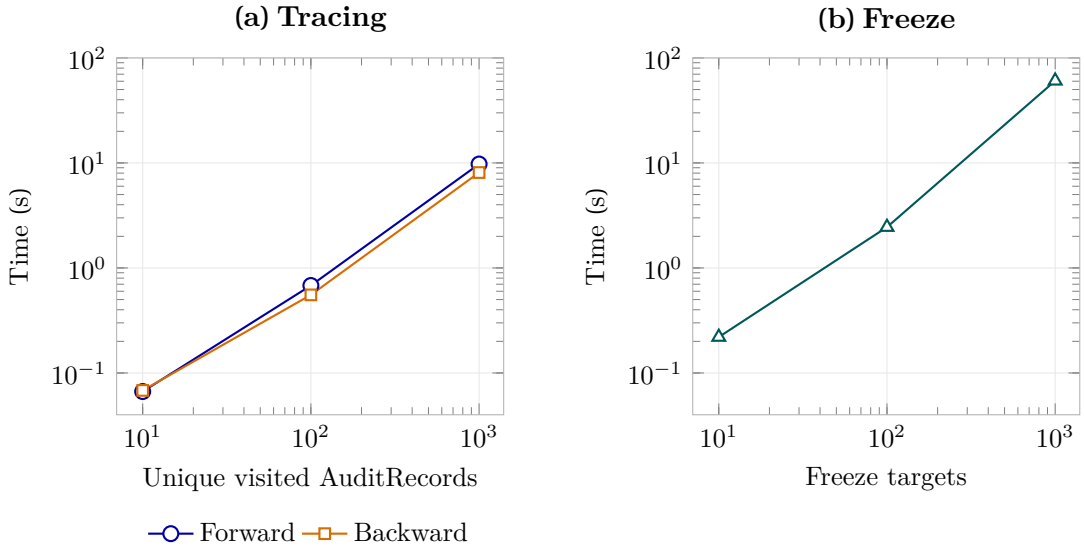


그림 2. 같은 공급망 시나리오의 Tracing · Freeze 시간.

Tracing

Visited AuditRecords	Forward (s)	Backward (s)
10	0.067	0.068
100	0.681	0.554
1,000	9.760	8.102

Freeze

Freeze targets	Time (s)
10	0.220
100	2.448
1,000	60.540
Gas per target:	**49,434**

3. zkTransfer와 Voucher 전달 Tradeoff 비용

실험을 진행한 이유
zkTransfer는 한 번에 Receiver Note를 만들지만, Voucher 전달은 Transfer와 Proceed 두 단계가 필요합니다.
Proof와 transaction이 하나씩 늘어나면 Gas가 당연히 증가할꺼라 생각해요.
zkTransfer를 조금 변경한 SharedSecret 구조로도 Traceability는 제공할 수 있어보이는데, 왜 Voucher가 필요할까요?

SharedSecret zkTransfer란?
기존 zkTransfer는 owner secret sk를 사용해 nf=H(cm, sk)를 계산합니다.
SharedSecret zkTransfer는 이를 shared secret 'o'로 계산하도록 변경한 구조입니다.
Sender와 Receiver가 opening을 공유하므로, Sender도 Receiver Note의 nf를 계산할 수 있습니다.
Receiver Note의 nf를 계산할 수 있으면, Receiver가 Note를 소비하는 시점을 식별할 수 있는 문제가 있습니다.
이후로는 sharedSecret zkTrasnfer를 zkTransfer^{2}로 표기하겠습니다.

Vocher를 도입함으로써 얻는 장점

1. Receiver 수령 전 Sender의 Recall
Receiver가 Proceed하기 전에는 Sender가 허용된 기한 안에서 Recall할 수 있습니다.
Sender가 직접 Note를 회수할 수 있다는 점이 중요합니다.

2. Sender에게는 후속 소비를 숨기고, Auditor에게는 추적을 허용
SharedSecret zkTransfer에서는 Sender도 Receiver Note의 nf를 계산해 후속 소비를 식별할 수 있습니다.
Voucher 방식에서는 Receiver가 Voucher를 소비한 시점을 알지만, 실제로 Note를 소비한 시점을 알 수 없습니다.
Sender는 Voucher의 nf를 계산할 수 있으므로, Sender가 Note를 소비 할 때 Receiver의 rvnf를 AuditRecord에 등록할 수 있습니다.
Auditor는 암호화된 rvnf를 복원해 Forward Tracing을 수행할 수 있습니다.

비교 조건
zkTransfer^1: Audit 없는 owner secret 기반 zkTrasnfer입니다.
zkTransfer^2: Audit을 지원하지만 Sender도 후속 소비를 식별할 수 있는 직접 전달입니다.
zkDPP: Audit을 포함한 Transfer → Recall 경로의 합계입니다.

표는 세 방식의 기능과 Gas를 비교합니다.
Recall 경로는 정상 수령 경로와 결과가 다르며, Audit 조건도 달라 Voucher만의 순수 추가 비용은 아닙니다.

전달 방식	Traceability	Privacy	Recall	총 Gas
zkTransfer^1	X	O	X	2,366,228
zkTransfer^2	O	△	X	2,724,242
zkDPP Transfer	O	O	O	5,362,498

△: Sender가 Receiver의 후속 소비 시점을 식별할 수 있습니다.

Azeroth의 zkTransfer와 총 Gas에서 크게 차이나는 이유
Azeroth의 zkTrasnfer의 총 Gas는 1,555,957로 실험한 zkTransfer^{1}와 총 Gas가 크게 차이납니다.
Azeroth의 zkTransfer에서는 Sender의 잔여분을 암호화 계정 잔액인 ENA에 보관합니다.
zkDPP에서 사용하는 zkTransfer^{1}는 Sender의 잔여분을 새로운 Note로 생성하여 Note Tree에 등록하고, address를 Sender로 지정합니다. Balance 방식과 UTXO 방식의 차이로 인해 총 Gas 소비량에서 차이가 발생합니다.

4. Process 입출력 cm 수에 따른 비용

실험을 진행한 이유
공정에 따라 여러 원료를 투입하거나 제품·부산물 생산이 많은 경우가 있습니다. Event별 실행 비용에서 Process는 2-to-1으로 작성되어 있습니다.
입력·출력 cm 수가 늘어나면, Process의 비용이 크게 증가하는 것은 아닐까요?

실험 목적·범위
공정의 입력·출력 개수에 따른 Constraints와 Gas의 변화 경향을 확인합니다.
모든 산업별 공정을 구현하는 대신, 기존 POC의 Sustainability Claim을 증명하기 위한 State로 범위를 제한해 경향성을 파악합니다.

통제 조건
입출력 개수의 영향을 분리하기 위해 입력 State의 합(320,30,231)과 출력 State의 합(300,30,261)을 고정합니다.
출력 N=1에서 입력 M을 늘리고 입력 M=2에서 출력 N을 늘립니다.

표·그래프 구성
입력 증가와 출력 증가의 영향을 구분하기 위해 두 묶음의 그래프를 사용합니다.
각 그래프의 왼쪽 축은 Constraints, 오른쪽 축은 Gas이며 색으로 구분합니다.
두 그래프의 축 범위는 같게 유지하고, 아래 표에 정확한 수치를 제공합니다.

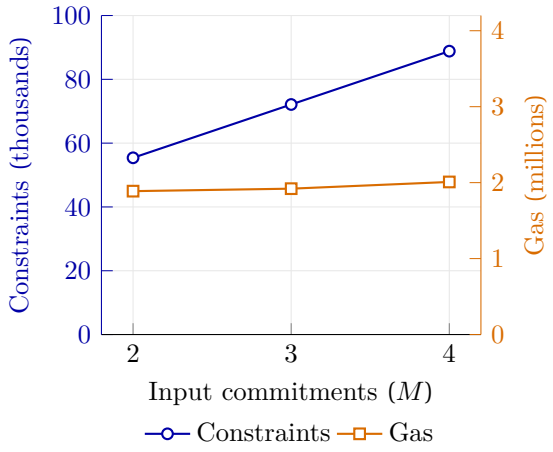


그림 3. 입력 수 증가 (N=1).

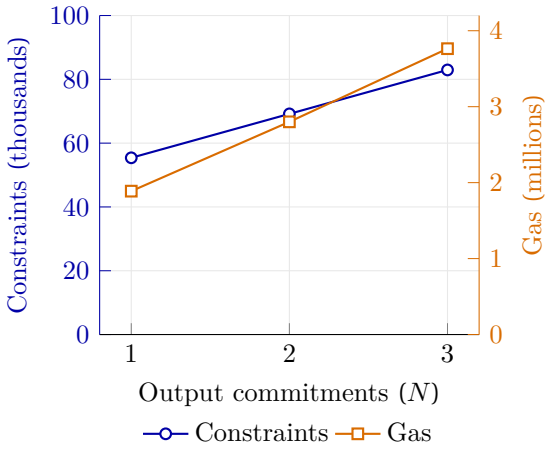


그림 4. 출력 수 증가 (M=2).

Outputs N=1

Input M	Constraints	Gas
2	55,402	1,887,935
3	72,116	1,920,611
4	88,832	2,009,039

Inputs M=2

Output N	Constraints	Gas
1	55,402	1,887,935
2	69,172	2,800,596
3	82,944	3,763,779

5. k-of-n Master Key 복구 비용

실험을 진행한 이유
Auditor는 Committee로부터 전달받은 k개의 share로 Master Key를 복구합니다.
k값에 따라서 복구 시간이 얼마나 늘어나나요?

측정 범위
Committee의 공개키와 위원별 public share는 사전에 검증해 보관합니다.
받은 k개 share의 검증, Lagrange 보간, 복구한 키의 공개키 일치 확인을 측정합니다.
키 생성·DKG·통신·승인 대기는 제외합니다.

비교 조건
Committee n=10을 고정하고 share k를 2·3·5·7·10개로 늘립니다.
각 case는 warm-up 후 공식 1회 측정하며, 내부 1,000회 복구의 평균 시간을 표시합니다.

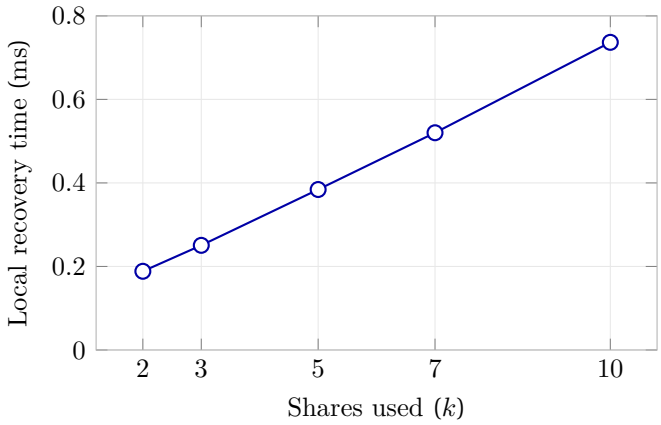


그림 5. Committee n=10에서 share k 증가에 따른 Master Key 복구 시간.

share k	복구 시간 (ms)
2	0.188
3	0.251
5	0.384
7	0.520
10	0.737

해석
k가 늘면 검증할 share 수와 Lagrange 보간의 연산량이 증가합니다.
복구식은 선택한 k개 share와 위원 ID를 사용하며, 전체 위원 수 n은 직접 포함하지 않습니다.

$$SK = \sum_{i \in Q} \lambda_i s_i \pmod{q}, \qquad \lambda_i = \prod_{\substack{j \in Q \\ j \neq i}} \frac{-x_j}{x_i - x_j} \pmod{q}, \qquad |Q| = k.$$

Q는 선택한 위원 집합, s_i는 share, x_i는 위원 ID이며 나눗셈은 modulo q의 역원 연산입니다.
n은 전체 위원 수와 $k \le n$ 조건을 정하지만, 고정된 Q의 복구 연산량을 늘리지 않습니다.